



INSTITUTO SUPERIOR TÉCNICO

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

ORGANIZAÇÃO DE COMPUTADORES

LEIC

Second Lab Assignment: System Modeling and Profiling

Version 1.1.0

2024/2025

1 Introduction

The goal of this assignment is twofold: (i) to determine the characteristics of a computer's caches, and (ii) to leverage the obtained knowledge about the caches in order to optimize the performance of a given program. For this task, the students will make use of a performance analysis tool to have direct access to hardware performance counters available on most modern microprocessors. The tool that will be used is the standard Application Programming Interface (API): PAPI [1].

In the rest of this section, we make a brief introduction to PAPI, and describe the targeted computer platform and the development environment. In Section 3, we describe the procedure for modeling the L1 and L2 caches of the targeted platform (Subsection 3.1), and provide a guide for analyzing the performance of a matrix-multiply code segment and optimizing it based on the characteristics of the L2 cache of the target architecture (Subsection 3.2).

1.1 Targeted Platform and Development Environment

IMPORTANT: This assignment must be performed on the computers of your lab classes room. These computers have similar hardware characteristics, and any of them can be used as a target platform. Note that, since this work is hardware-dependent, conducting it on a computer with different hardware characteristics could produce unexpected results, and hence invalidating your work. This means you should always use the same lab. If you are an Alameda student, you can access the specific lab computer you want (see <https://welcome.rnl.tecnico.ulisboa.pt/#labs-access>).

To properly setup the development environment, it is necessary to obtain the PAPI library and a set of auxiliary program files. This material can be found in the package `lab1_kit.zip`, which can be downloaded from the course website. After downloading and uncompressing this package on any of the lab classes' computers, PAPI must be built. To this end, change directories to the location of the PAPI source code: folder `papi-X.X.X/src`. Compile the code by issuing the commands: `./configure`, and `make`. This operation will produce a set of helper tools located in directory `src/utils/` and create the PAPI library `papilib.a`. The tool `papi_avail`, in particular, is useful to determine the PAPI events supported on the target platform. The library will be linked to the auxiliary programs presented in the following sections.

2 Exercise

To help determining the characteristics of the labs computer's caches, the following exercises will help you estimate cache parameters from small C applications.

The first step to get acquainted with the procedure is to determine only the size of the cache using a small C application on a (known) machine, such as the code you have analyzed on lab exercise VI.3. This C code, is a simplified version of the following programs in this assignment. Basically, it iterates over an array to determine the cache size.

To guarantee that you measure the time accurately, please use the source code available in the lab kit (file spark.c).

In order to perform the evaluation you should go to your lab in order to access the cache size by running the application there. You may want to repeat the evaluation of the elapsed time a few times to achieve statistical significance. You should table the relevant results for different cache sizes on the response sheet and make a conclusion regarding the cache size. You can calculate more measures before the output, examine the final part of the source code file.

1. What is the cache capacity of the computer you tested? Please justify.

To discover the other cache parameters, you're going to modify the C application, so that it generates different data access patterns. Please spend a few minutes analyzing the modifications to the source code.

```
for(size_t cache_size = CACHE_MIN; cache_size < CACHE_MAX; cache_size = 2*cache_size) {
    for(size_t stride = 1; stride <= cache_size/2; stride = 2*stride){
        limit = cache_size - stride + 1;
        for(ssize_t i = 10 * stride; i > 0; i--) {
            for(index = 0; index < limit; index += stride) {
                array[index] = array[index] + 1;
            }
        }
    }
}
```

The meaning of each variable is the following:

array[] an arbitrary large array that will be repeatedly accessed to measure the cache miss pattern;

cache_size value of the cache size under test; all cache sizes given by integer powers of 2, between `CACHE_MIN = 8kB` and `CACHE_MAX = 64kB` should be considered;

stride states how many entries are being skipped at each access; for example, if the stride is 4, entries 0, 4, 8, 12, ... in the array are being accessed, while entries 1, 2, 3, 5, 6, 7, 9, 10, 11, ... are skipped;

limit the largest address that will be accessed for the cache size and access pattern under test;

repeat denotes the number of times that each access pattern will be repeated in array.

The execution time for this code segment on this machine yield the chart depicted in Figure 1, by varying the adopted value for the *stride* parameter and for different array sizes, defined between `ARRAY_MIN = 4kB` and `ARRAY_MAX = 4MB`.

2. What is the cache capacity of the computer?
3. What is the size of each cache block?
4. What is the L1 cache miss penalty time?

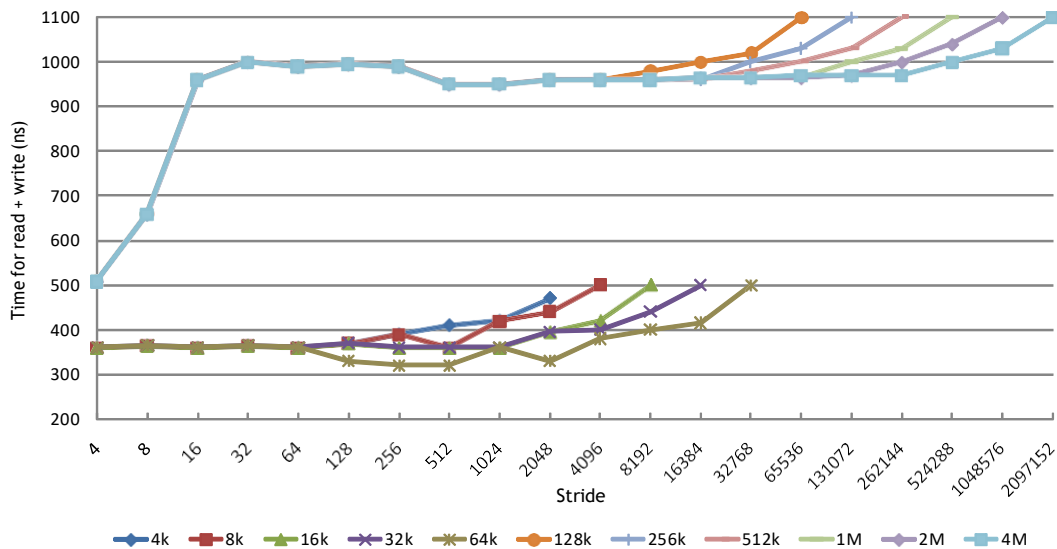


Figure 1: Variation of the cache access time with the adopted *stride* value for different array sizes.

3 Procedure

3.1 Modeling Computer Caches

In the first part of this assignment, the goal is to model the characteristics of the L1 data cache and L2 cache of the targeted computer platform. Next, we provide instructions for performing this analysis.

Use the forms at the end to answer the questions below.

3.1.1 Modeling the L1 Data Cache

The methodology to experimentally model the L1 data cache consists in considering the total amount of data cache misses during the execution of the following code sequence of program `cm1.c`, similar to the program in Section 2. This program can be found in the package `lab1_kit.zip`.

```
for(array_size=ARRAY_MIN; array_size < ARRAY_MAX; array_size=array_size*2)
    for(stride=1; stride <= array_size/2; stride=stride*2){
        limit = array_size - stride + 1;
        for(repeat=0; repeat<=200*stride; repeat++)
            for(index=0; index<limit; index+=stride)
                x[index] = x[index] + 1;
    }
}
```

- Change to directory `cm1/`, in the package `lab1_kit.zip`, and analyze the code of the program `cm1.c`. Identify its source code with the program described above.
What are the processor events that will be analyzed during its execution? Explain their meaning.
- Compile the program `cm1.c` using the provided `Makefile` and execute `cm1`. Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).

NOTE: A fast sketch of these plots can be drawn in your computer by running the following commands:

```
./cm1 > cm1.out
./cm1_proc.sh
```

NOTE 2: You can draw these tables and plots on your computer, print, and attach to the report. You do not have to

fill them by hand on the printed report.

NOTE 3: You may need to mark the script as executable before being able to run it.

c) By analyzing the obtained results:

- Determine the **size** of the L1 data cache. Justify your answer.
- Determine the **block size** adopted in this cache. Justify your answer.
- Characterize the **associativity set size** adopted in this cache. Justify your answer.

3.1.2 Modeling the L2 Cache

In this part of the assignment, the goal is to experimentally model the characteristics of the L2 cache of the targeted computer platform. To analyze the computer's L2 cache, we will use the same methodology that was introduced in the previous section to model the L1 data cache.

- a) Modify the program `cm1.c` in order to analyze the characteristics of the L2 cache. (Hint: use the event `PAPI_L2_DCM`.) Describe and justify the changes introduced in this program.
- b) Compile the program `cm1.c`, execute `cm1`, and plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.
- c) By analyzing the obtained results:
 - Determine the **size** of the L2 cache. Justify your answer.
 - Determine the **block size** adopted in this cache. Justify your answer.
 - Characterize the **associativity set size** adopted in this cache. Justify your answer.

3.2 Profiling and Optimizing Data Cache Accesses

Often, programmers wishing to improve their programs' performance focus their attention on how the programs affect the computer's caches. In the following, it will be analyzed how simple code changes can help to improve that performance for a matrix multiplication application.

Consider a simple matrix multiplication application, operating on two square matrices of $N \times N$ 16-bit integer elements, with $N = 1024$. From a mathematical point of view, given two matrices **A** and **B**, with elements a_{ij} and b_{ij} such that $0 \leq i, j < N$, the product matrix **C** is defined as:

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik}b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{i(N-1)}b_{(N-1)j} \quad (1)$$

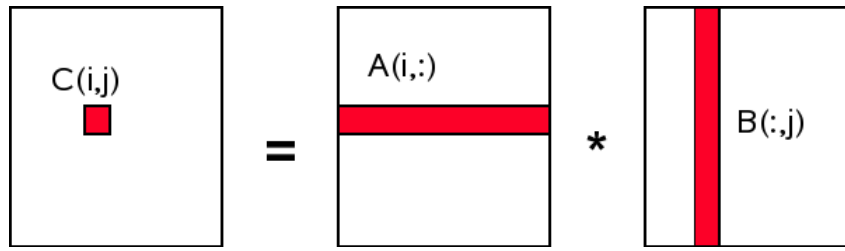


Figure 2: Straightforward matrix multiplication.

3.2.1 Straightforward implementation

A straight-forward C implementation of Eq. 1 can look like this:

```
for (i = 0; i < N; ++i) {
    for (j = 0; j < N; ++j) {
        for (k = 0; k < N; ++k) {
            res[i][j] += mul1[i][k] * mul2[k][j];
        }
    }
}
```

The two input matrices are `mul1` and `mul2`. The result matrix `res` is assumed to be initialized to all zeroes.

The provided program `mm1.c` includes this code sequence and all the necessary initialization steps, as well as the set of statements that are required in order to profile its execution using the PAPI toolbox.

- Change to directory `mm1/` and analyze the code of the program `mm1.c`. Identify its source code with the program described above.
What is the total amount of memory that is required to accommodate each of these matrices?
- Compile the source file `mm1.c` using the provided `Makefile` and execute it. Fill the table with the obtained data.
- Evaluate the resulting L1 data cache *Hit-Rate*.

3.2.2 First Optimization: Matrix transpose before multiplication [2]

By analyzing the obtained results, it can be observed that such a straightforward implementation suffers from a severe penalty in what concerns the amount of L2 cache misses resulting from its access pattern. In fact, while `mul1` matrix is accessed sequentially, the inner loop advances the row number of `mul2` (see Fig. 2), meaning successive accesses to far away memory positions.

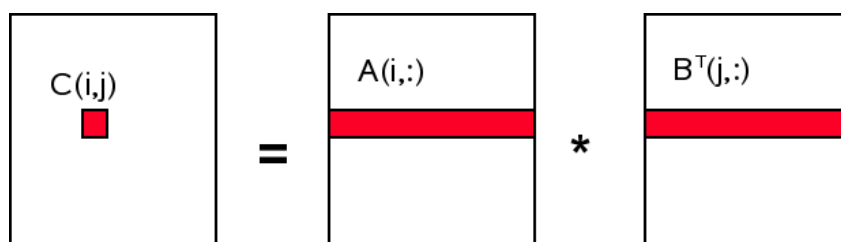


Figure 3: Transposed matrix multiplication.

One possible remedy to attenuate such problem is based on matrix transposition. In fact, since each matrix element is accessed multiple times, it might be worthwhile to rearrange (“transpose,” in mathematical terms) the second matrix `mul2` before using it (see Fig. 3):

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{jk}^T = a_{i1} b_{j1}^T + a_{i2} b_{j2}^T + \dots + a_{i(N-1)} b_{j(N-1)}^T \quad (2)$$

After the preliminary transposition step, both matrices may be iterated sequentially. As far as the C code is concerned, it now looks like this:

```
int16_t tmp[N][N];

// transposition
for (i = 0; i < N; ++i) {
    for (j = 0; j < N; ++j) {
        tmp[i][j] = mul2[j][i];
    }
}

// multiplication
for (i = 0; i < N; ++i) {
    for (j = 0; j < N; ++j) {
        for (k = 0; k < N; ++k) {
            res[i][j] += mul1[i][k] * tmp[j][k];
        }
    }
}
```

Variable `tmp` is a temporary array to store the transposed matrix.

One direct consequence of this optimization is that it now requires additional accesses to the data memory. Hopefully, this extra cost can be easily recovered, since the 1024 non-sequential accesses per column are usually much more expensive.

- a) Change to directory `mm2/` and analyze the code of the program `mm2.c`. Identify its source code with the program described above. Compile this program using the provided `Makefile` and execute it.

Fill the table with the obtained data.

- b) Evaluate the resulting L1 data cache *Hit-Rate*.
- c) Change the code in the program `mm2.c` in order to include the matrix transposition in the execution time. Compile this program using the provided `Makefile` and execute it.

Fill the table with the obtained data.

Comment on the obtained results when including the matrix transposition in the execution time.

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedups.

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

Despite the good results that may be obtained with the matrix transposition method, in many applications this approach can not be applied, either because the matrix is too large or the available memory is too small. Hence, other alternatives, which do not require the extra copy procedure, should be studied.

The search for an alternative processing scheme should start with a close examination of the involved math and the operations performed by the original implementation. Trivial math knowledge shows that the order of the several additions to obtain each element of the result matrix is irrelevant, as long as

each addend appears exactly once. This understanding will lead to solutions which reorder the additions performed in the inner loop of the original code.

According to the original algorithm, the adopted order to access the elements of matrix `mul2` is: (0,0), (1,0), ... , (N-1,0), (0,1), (1,1), Although the elements (0,0) and (0,1) are in the same cache line, by the time the inner loop completes one round, this cache line has long been evicted. For this example, each round of the inner loop requires, for each of the three matrices, 1024 cache lines, which is much more than what is available in most processors' caches.

One possible solution is to simultaneously handle more than one iteration of the middle loop, while executing the inner loop. In this case, several values which are guaranteed to be in cache will be used, thus contributing to a reduction of the L2 cache miss-rate. Hence, to maximize the speedup provided by this technique, it is necessary to adapt the dimension of the sub-matrix under processing to the cache block size, by taking into account the size of each matrix element. As a hypothetical example, considering that a `short` operand occupies 2-Bytes, this means that a 64-Byte cache block will accommodate 32 matrix elements, thus defining the optimal size for the sub-matrix line to be 32 (see Fig. 4).

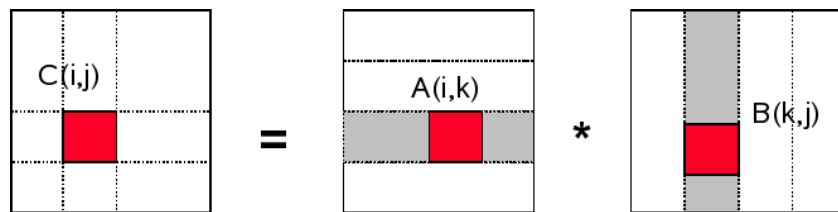


Figure 4: Blocked matrix multiplication.

As far as the C code is concerned, it now looks like this:

```
#define SUB_MATRIX_SIZE (CACHE_LINE_SIZE / sizeof (short))

for (i = 0; i < N; i += SUB_MATRIX_SIZE) {
    for (j = 0; j < N; j += SUB_MATRIX_SIZE) {
        for (k = 0; k < N; k += SUB_MATRIX_SIZE) {
            for (i2 = 0, rres = &res[i][j], rmul1 = &mul1[i][k];
                 i2 < SUB_MATRIX_SIZE;
                 ++i2, rres += N, rmul1 += N) {
                for (k2 = 0, rmul2 = &mul2[k][j]; k2 < SUB_MATRIX_SIZE; ++k2, rmul2 += N) {
                    for (j2 = 0; j2 < SUB_MATRIX_SIZE; ++j2) {
                        rres[j2] += rmul1[k2] * rmul2[j2];
                    }
                }
            }
        }
    }
}
```

The most visible change is that the code has six nested loops now. The outer loops iterate with intervals of `SUB_MATRIX_SIZE` (the cache line size `CACHE_LINE_SIZE` divided by `sizeof(short)`). This divides the matrix multiplication in several smaller problems which can be handled with more cache locality. The inner loops iterate over the missing indexes of the outer loops. There are, once again, three loops. The `k2` and `j2` loops are in a different order. This is done because, in the actual computation, only one expression depends on `k2` but two depend on `j2`.

- a) Change to directory `mm3/` and analyze the code of the program `mm3.c`. Identify its source code with the program described above.

Change the program source code in order to comply the algorithm parameterization (sub-matrix line size) with the block size (CLS) that was determined in Section 3.1.

How many matrix elements can be accommodated in each cache line?

- b) Compile this program using the provided `Makefile` and execute it. Fill the table with the obtained data.

- c) Evaluate the resulting L1 data cache *Hit-Rate*.
- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.
- e) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup. If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement? (Hint: study the hit-rates of the L2 cache for both implementations; You may use the following PAPI events PAPI_L2_DCH (or PAPI_L2_DCM) and PAPI_L2_DCA. Run `papi_avail` to check for available events and understand their meaning.)

References

- [1] Performance Application Programming Interface (PAPI). Webpage. "<http://icl.cs.utk.edu/papi>", December 2008.
- [2] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., November 2007.
- [3] *PAPI User's Guide*.
- [4] *PAPI Programmer's Reference*.

Second Lab Assignment: System Modeling and Profiling

STUDENTS IDENTIFICATION:

Number:	Name:
106937	André Melo
106900	Gonçalo Aleixo
106642	Pedro Silveira

2 Exercise

Please justify all your answers with values from the experiments.

1. What is the cache capacity of the computer you used (please write the workstation name)?

Array Size	4kB	8kB	16kB	32kB	64kB	128kB
t2-t1	0.000820	0.001699	0.003399	0.006804	0.014600	0.029207
# accesses a[i]	409600	812900	1638400	3276800	6553600	13107200
# mean access time	2.001953 ns	2.073975 ns	2.074585 ns	2.076416 ns	2.227783 ns	2.228317 ns

Usámos o computador Lab6p1. Analisando a tabela acima, é possível observar que o mean access time até 32kB é constante, praticamente, quando o size aumenta para 64kB, o mean access time aumenta consideravelmente, o que significa que a miss rate também aumenta, o que significa que existe um aumento da frequência dos acessos à main memory. Logo, o tamanho da cache do computador é de 32kB.

Consider the data presented in Figure 1. Answer the following questions (2, 3, 4) about the machine used to generate that data.

2. What is the cache capacity?

Devido ao facto de haver um aumento significativo do tempo de read/write quando se transita de uma array de 64kB para uma array de 128kB, existe um número maior de acessos à main memory na de 128kB. Logo, o tamanho da cache é de 64kB.

3. What is the size of each cache block?

Quando se testa com um tamanho de array superior a 64kB (o tamanho da cache), há aumentos acentuados nos tempos de leitura/escrita ao aumentar o stride de 4 para 8 e de 8 para 16. Isto acontece porque um stride maior significa que os acessos atravessam as linhas da cache mais rapidamente, o que leva a misses mais frequentes. No entanto, esses aumentos estagnam para strides de tamanho 16 ou superiores, indicando que o stride agora é do mesmo tamanho ou maior que a linha da cache, o que faz com que nos encontremos numa nova linha em cada acesso (miss rate aproximadamente 100%). Assim, conclui-se que o tamanho do bloco da cache é de 16 bytes.

4. What is the L1 cache miss penalty time?

É possível calcular o miss penalty de L1 comparando o execution time total quando a miss rate é aproximadamente 0% com quando é aproximadamente 100%.
A miss rate é aproximadamente 0% para tamanhos de arrays menores que o tamanho da cache, ou seja, quando o access time está em torno de 360ms. É aproximadamente 100% para arrays de tamanhos maiores, juntamente com um stride maior ou igual a 16 bytes, ou seja, quando o access time está em torno de 1000ms. Portanto, o miss penalty é $1000 - 360 = 640$ ms.

3 Procedure

3.1.1 Modeling the L1 Data Cache

- a) What are the processor events that will be analyzed during its execution? Explain their meaning.

No programa cm1.c vai ser analisado PAPI_L1_DCM, ou seja, “L1 Data Cache Misses”. Logo, iremos estar a analisar a quantidade de vezes que ocorrem data cache misses, isto é, quantas vezes tentámos dar fetch em data que não estava na cache L1, durante a execução do programa.

- b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).

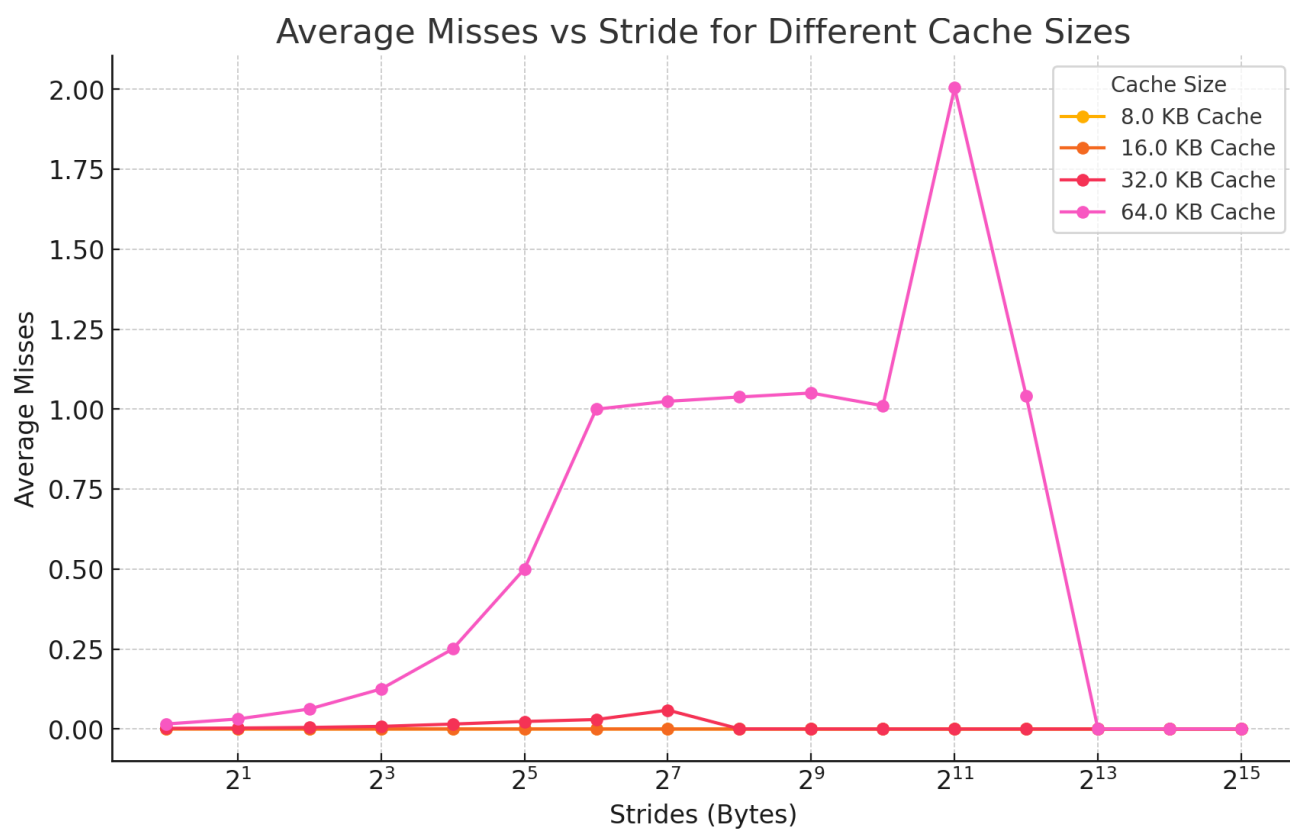
Note that, you may fill these tables and graphics (as well as the following ones in this report) on your computer and submit the printed version.

Array Size	Stride	Avg Misses	Avg Cycl Time
8kByte	1	0,000244	0,002689
	2	0,000147	0,002799
	4	0,000066	0,002784
	8	0,000052	0,002689
	16	0,000058	0,002727
	32	0,000059	0,002733
	64	0,000053	0,002436
	128	0,000003	0,002149
	256	0,000002	0,002016
	512	0,000015	0,002010
	1024	0,000011	0,001948
	2048	0,000001	0,002040
	4096	0,000009	0,002090

Array Size	Stride	Avg Misses	Avg Cycl Time
32kByte	1	0,002101	0,002217
	2	0,003135	0,002202
	4	0,004851	0,002183
	8	0,008253	0,002103
	16	0,015435	0,001982
	32	0,023543	0,002002
	64	0,029597	0,001901
	128	0,059047	0,002224
	256	0,000318	0,00218
	512	0,000165	0,002059
	1024	0,000075	0,002002
	2048	0,000036	0,002045
	4096	0,000022	0,002188
	8192	0,000009	0,002173
	16384	0,000003	0,002078

Array Size	Stride	Avg Misses	Avg Cycl Time
16kByte	1	0,000278	0,002261
	2	0,000187	0,002238
	4	0,000178	0,002221
	8	0,000183	0,002156
	16	0,000219	0,002214
	32	0,000188	0,002226
	64	0,000175	0,002171
	128	0,000085	0,002180
	256	0,000054	0,002061
	512	0,000032	0,001999
	1024	0,000015	0,001986
	2048	0,000015	0,002083
	4096	0,000013	0,002179
	8192	0,000006	0,002078

Array Size	Stride	Avg Misses	Avg Cycl Time
64kByte	1	0,015636	0,001983
	2	0,031268	0,001906
	4	0,062575	0,002134
	8	0,125109	0,002206
	16	0,250265	0,002179
	32	0,500584	0,002046
	64	0,999786	0,001863
	128	1,024432	0,001873
	256	1,037993	0,001896
	512	1,050386	0,001926
	1024	1,010487	0,001881
	2048	2,004544	0,005699
	4096	1,040486	0,005104
	8192	0,000002	0,002199
	16384	0,000008	0,002184
	32768	0,000002	0,002086



c) By analyzing the obtained results:

- Determine the **size** of the L1 data cache. Justify your answer.

Podemos determinar o tamanho da cache ao olhar para o plot e observar qual é o maior size de array que tem aproximadamente 0 como média de misses. Logo, a linha vermelha (32kBytes) é o maior tamanho que cumpre o requisito, mesmo tendo um pequeno bump do stride 2^4 até ao stride 2^7 , que pode ser ignorado devido ao facto da cache ser ocupada por outra data do programa. Logo, a cache tem 32kBytes.

- Determine the **block size** adopted in this cache. Justify your answer.

Quando o stride é menor que 64Bytes, a miss rate aumenta, ou seja, existem alguns acessos que dão hit na cache, o que significa que estão no mesmo bloco que os acessos anteriores. Quando o plot estagna, significa que saímos dos limites do bloco, isto é, os acessos sequenciais estão a ser feitos a blocos diferentes. Logo, observa-se que o tamanho do bloco são 64Bytes.

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

Para determinar o tamanho do conjunto de associatividade, precisamos analisar em que stride a miss rate volta a ser zero para a maior cache (especificamente, a linha vermelha representando 64KiB). Como o stride determina quantos blocos únicos de dados da matriz estamos a aceder, podemos ver que, para um stride de 2^{15} (o máximo para este tamanho de matriz), estamos apenas a aceder a 2 blocos diferentes. Se a cache for pelo menos 2-way associativa, a miss rate deve cair para perto de zero. Repetindo esta análise para strides menores, observamos que, para 2^{14} e 2^{13} , a miss rate também se aproxima de zero. Isto indica que a cache é pelo menos 4-way ou 8-way associativa. No entanto, quando reduzimos o stride para 2^{12} , este padrão deixa de se verificar, sugerindo que ultrapassámos o limite de associatividade da cache. Logo, o associativity set size é 8.

3.1.2 Modeling the L2 Cache

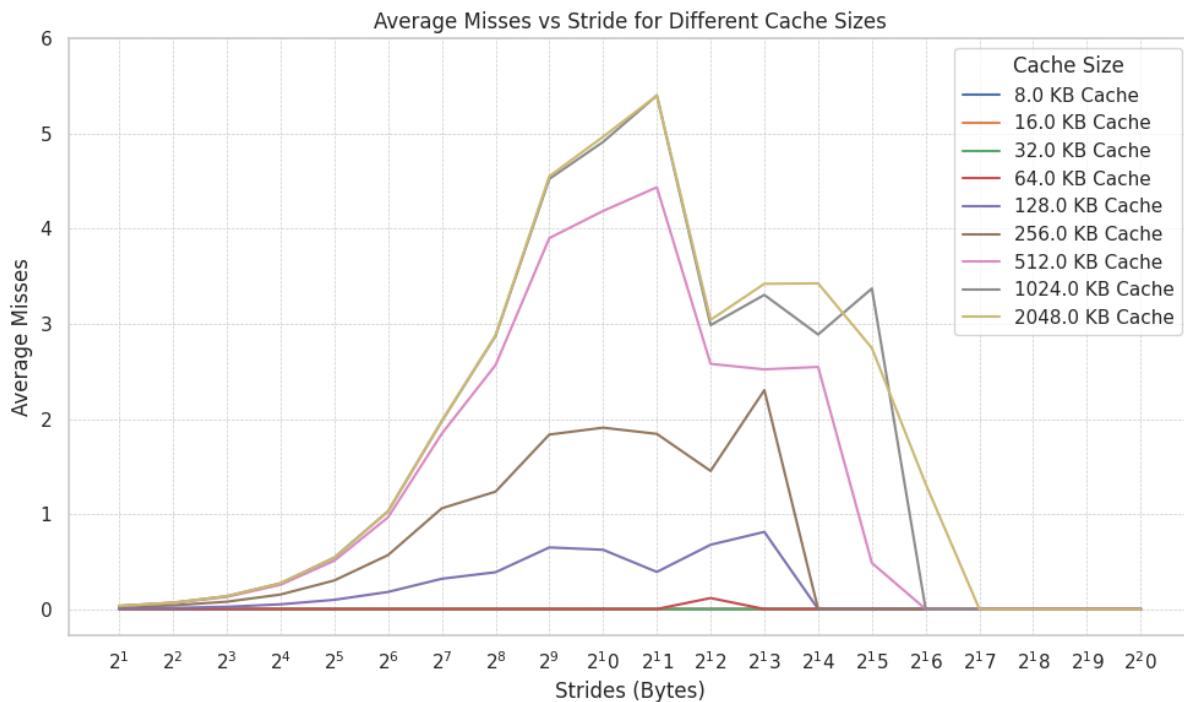
- a) Describe and justify the changes introduced in this program.

Alterámos no programa o PAPI event e a CACHE_MAX.

Primeiramente, alterámos o PAPI event de PAPI_L1_DCM para PAPI_L2_DCM, isto porque queremos ver a miss rate da cache L2.

Em segundo lugar, aumentámos o valor de CACHE_MAX para 2MB ($2 \times 1024 \times 1024$), devido ao facto de L2 ser bastante maior que L1, ou seja, precisamos de arrays maiores para ultrapassar o tamanho de L2.

- b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.



c) By analyzing the obtained results:

- Determine the **size** of the L2 cache. Justify your answer.

Ao olhar para o plot, é possível observar um salto significativo na miss rate de 128kB para 256kB e de 256kB para 512kB. O salto de 256kB para 512kB é substancialmente maior, o que significa que excedemos o tamanho da cache e já não conseguimos manter a array toda em L2. Logo, o tamanho de L2 é de 256kB.

- Determine the **block size** adopted in this cache. Justify your answer.

Quando o stride é menor que 512 Bytes, a miss rate aumenta significativamente, ou seja, há acessos a dar hit na cache, o que significa que estão no mesmo bloco de acessos anteriores. Quando o plot estagna consideravelmente, saímos dos limites do bloco, isto é, acessos sequenciais são feitos em diferentes blocos. Logo, o tamanho do bloco é 512 Bytes.

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

Assim como em L1, é possível determinar o associativity set size ao observar em que stride é que o miss rate fica 0 para a array com o maior tamanho (explicação completa na resposta a L1). Como a array de 2048kB volta a 0 quando o stride é 2^{17} , temos que o associativity set size é 16.



3.2 Profiling and Optimizing Data Cache Accesses

3.2.1 Straightforward implementation

- a) What is the total amount of memory that is required to accommodate each of these matrices?

Analizando o código é possível observar que cada matriz está completa com integer elements de 16-bits, ou seja, 2 bytes por elemento. Cada matriz tem 265144 elementos (como a variável N está definida como 512 e cada matriz está inicializada como m[N][N], portanto $512 \times 512 = 265.144$ elementos). Assim, cada matriz ocupa $2 \times 265.144 = 524.288$ KiB.

- b) Fill the following table with the obtained data.

Total number of L1 data cache misses	135.136425×10^6
Total number of load / store instructions completed	536.8721×10^6
Total number of clock cycles	602.980050×10^6
Elapsed time	0.200994 seconds

- c) Evaluate the resulting L1 data cache *Hit-Rate*:

hitRate = 1 - missRate = 1 - (L1DataCacheMisses / LoadStoreInstructions) =
1 - (135.136425 / 536.8721) = 0.7482893504803099
74.83% de data cache hit, as matrizes não cabem todas na L1.

3.2.2 First Optimization: Matrix transpose before multiplication [2]

- a) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.215548×10^6
Total number of load / store instructions completed	536.872075×10^6
Total number of clock cycles	535.201845×10^6
Elapsed time	0.178402 seconds

- b) Evaluate the resulting L1 data cache *Hit-Rate*:

hitRate = 1 - missRate = 1 - (L1DataCacheMisses / LoadStoreInstructions) =
1 - (4.215548 / 536.872075) = 0.99215.
99.215% de data cache hit

- c) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.483040×10^6
Total number of load / store instructions completed	537.396517×10^6
Total number of clock cycles	545.705244×10^6
Elapsed time	0.181902 seconds

Comment on the obtained results when including the matrix transposition in the execution time:

hitRate = 1 - missRate = 1 - (L1DataCacheMisses / LoadStoreInstructions) =
1 - (4.483040 / 545.705244) = 0.99178486912
99.178% de data cache hit
Ao comparar ambos os resultados, é possível chegar à conclusão que incluir a transposition da matriz no tempo de execução não altera os resultados significativamente. Isto provavelmente acontece devido ao facto da transposition da matriz ($O(N^2)$) ser mais rápida e um processo menos intensivo, computacionalmente, do que a multiplicação de matriz ($O(N^3)$)

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedups.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm2}} - \text{HitRate}_{\text{mm1}}: 0.2434955 = 24.3\%$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm1}} / \#\text{Clocks}_{\text{mm2}}: 1.10486$
$\text{Speedup}(\text{Time}) = \text{Time}_{\text{mm1}} / \text{Time}_{\text{mm2}}: 1.10494$
Comment: Foi obtido um Speedup (bastante pequeno) de 1.105 em comparação com a implementação original, embora a hit rate tenha aumentado significativamente para mais 24,35%. Isto pode ter acontecido devido ao facto de alguns acessos terem sido armazenados na cache L2 em vez de na cache L3 ou na main memory, reduzindo a miss penalty. Considerando o pequeno Speedup obtido e os requisitos adicionais de memória que esta otimização exige (a main memory tem agora de conter outra matriz NxN), esta otimização pode não ser desejável em sistemas com menor capacidade de memória.

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

- a) How many matrix elements can be accommodated in each cache line?

$\text{elements_per_cache_line} = \text{l1_block_size} * \text{l1_associativity_set_size} / 2 = 64 * 8 / 2 = 256$

- b) Fill the following table with the obtained data.

Total number of L1 data cache misses	2.508359×10^6
Total number of load / store instructions completed	537.945168×10^6
Total number of clock cycles	232.169466×10^6
Elapsed time	0.077390 seconds

- c) Evaluate the resulting L1 data cache *Hit-Rate*:

$\text{hitRate} = 1 - \text{missRate} = 1 - (\text{L1DataCacheMisses} / \text{LoadStoreInstructions}) =$
 $1 - (2.508359 / 537.945168) = 0.99215.$
99.534% de data cache hit

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm1}}: \mathbf{0.2470506495196901 (24.7\%)}$
$\text{Speedup}(\# \text{Clocks}) = \# \text{Clocks}_{\text{mm1}} / \# \text{Clocks}_{\text{mm3}}: \mathbf{2.59679}$
Comment: Foi atingido um Speedup de 2,6 dividindo a matriz em várias sub-matrizes que cabem precisamente dentro de um block da cache (assegurando que todos os dados necessários para o cálculo de uma dada matriz estão (quase sempre) presentes na cache L1 durante esse cálculo, tirando partido da localidade espacial), aumentando assim a hit rate em mais 24,7%. Não há desvantagens significativas na exploração desta otimização, para além do facto de o código agora ser ligeiramente mais complicado de ler do que a implementação original.

- e) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup. If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement? (Hint: study the hit-rates of the L2 cache for both implementations;)

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm2}}: -0.00036513088 = (-0.0365\%)$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm2}}/\#\text{Clocks}_{\text{mm3}}: 2.34915$
Comment: blocked_l2_dch = 3.19341 blocked_l2_dca = 5.77563 blocked_l2_hit_rate = blocked_l2_dch / blocked_l2_dca = 0.552911111 transpose2_l2_dch = 0.018964 transpose2_l2_dca = 5.184601 transpose2_l2_hit_rate = transpose2_l2_dch / transpose2_l2_dca = 0.00365775496 Ao comparar a transposição de matrizes e as otimizações de multiplicação de matrizes bloqueadas, descobrimos uma aceleração de 2.35. Isso ocorre apesar do facto de que a hit rate delta L1 ser negativa (-0,0365%). A hit rate delta L2 observada, no entanto, compensa a da cache L1, uma vez que a otimização da blocked multiplication tem uma hit rate L2 superior em 54.92% (blocked_l2_hit_rate - transpose2_l2_hit_rate) à da última otimização, resultando num miss penalty inferior para os erros L1. Assim, esta otimização não deve aumentar visivelmente os requisitos de memória (em contraste com a otimização da transposição de matrizes) e, por isso, pode provavelmente ser utilizada em vez da implementação original sem quaisquer requisitos de memória adicionais.

3.2.3 Comparing results against the CPU specifications

Now that you have characterized the cache on your lab computer, you are going to compare it against the manufacturer's specification. For this you can check the device's datasheet, or make use of the command `lscpu`. Comment the results.

Comparando as especificações do processador, obtidas com `lscpu -C` e `getconf -a | grep CACHE`, concluímos que na L1, o cache size, o block size e a associative size estavam corretos. O nosso cache size da L2 também estava correto, mas obtivemos um cache block size de 512 e associative set size de 16, enquanto o tamanho real dos dois é 64 e 4, respectivamente. Isto significa que podemos ter obtido dados incorretos, analisado os gráficos incorretamente, ou que o CPU tem algum tipo de otimização que não estamos a contabilizar, ou alguma combinação dos três.

A PAPI - Performance Application Programming Interface

The PAPI project [1] specifies a standard Application Programming Interface (API) for accessing hardware performance counters available in most modern microprocessors. These counters exist as a small set of registers that count *Events*, defined as occurrences of specific signals related to the processor's function (such as cache misses and floating point operations), while the program executes on the processor. Monitoring these events may have a variety of uses in the performance analysis and tuning of an application, since it facilitates the correlation between the source/object code structure and the efficiency of the actual mapping of such code to the underlying architecture. Besides performance analysis, and hand tuning, this information may also be used in compiler optimization, debugging, benchmarking, monitoring and performance modeling.

PAPI has been implemented on a number of different platforms, including: Alpha; MIPS R10K and R12K; AMD Athlon and Opteron; Intel Pentium II, Pentium III, Pentium M, Pentium IV, Itanium 1 and Itanium 2; IBM Power 3, 4 and 5; Cell; Sun UltraSparc I, II and II, etc.

Although each processor has a number of events that are native to that specific architecture, PAPI provides a software abstraction of these architecture-dependent *Native Events* into a collection of *Preset Events*, also known as *predefined events*, that define a common set of events deemed relevant and useful for application performance tuning. These events are typically found in many CPUs that provide performance counters. They give access to the memory hierarchy, cache coherence protocol events, cycle and instruction counts, functional unit, and pipeline status. Hence, preset events may be regarded as mappings from symbolic names (PAPI preset name) to machine specific definitions (native countable events) for a particular hardware resource. For example, Total Cycles (in user mode) is mapped into PAPI_TOT_CYC. Some presets are derived from the underlying hardware metrics. For example, Total L1 Cache Misses (PAPI_L1_TCM) is the sum of L1 Data Misses and L1 Instruction Misses on a given platform. The list of preset and native events that are available on a specific platform can be obtained by running the commands `papi_avail` and `papi_native_avail`, both provided by the papi source distribution.

Besides the standard set of events for application performance tuning, the PAPI specification also includes both a high-level and a low-level sets of routines for accessing the counters. The high level interface consists of eight functions that make it easy to get started with PAPI, by simply providing the ability to start, stop, and read sets of events. This interface is intended for the acquisition of simple but accurate measurement by application engineers [3, 4]:

- `PAPI_num_counters` – get the number of hardware counters available on the system;
- `PAPI_flops` – simplified call to get Mflops/s (floating point operation rate), real and processor time;
- `PAPI_ipc` – gets instructions per cycle, real and processor time;
- `PAPI_accum_counters` – add current counts to array and reset counters;
- `PAPI_read_counters` – copy current counts to array and reset counters;
- `PAPI_start_counters` – start counting hardware events;
- `PAPI_stop_counters` – stop counters and return current counts.

The following is a simple code example of using the high-level API [3, 4]:

```

#include <papi.h>

#define NUM_FLOPS 10000
#define NUM_EVENTS 1

int main(){
    int Events[NUM_EVENTS] = {PAPI_TOT_INS};
    long_long values[NUM_EVENTS];

    /* Start counting events */
    if (PAPI_start_counters(Events, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    do_some_work();

    /* Read the counters */
    if (PAPI_read_counters(values, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    printf("After reading the counters: %lld\n", values[0]);

    do_some_work();

    /* Add the counters */
    if (PAPI_accum_counters(values, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    printf("After adding the counters: %lld\n", values[0]);

    do_some_work();

    /* Stop counting events */
    if (PAPI_stop_counters(values, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    printf("After stopping the counters: %lld\n", values[0]);
}

```

Possible output:

```

After reading the counters: 441027
After adding the counters: 891959
After stopping the counters: 443994

```

The fully programmable low-level interface provides more sophisticated options for controlling the counters, such as setting thresholds for interrupt on overflow, as well as access to all native counting modes and events. Such interface is intended for third-party tool writers or users with more sophisticated needs.

The PAPI specification also provides access to the most accurate timers available on the platform in use. These timers can be used to obtain both real and virtual time on each supported platform: the real time clock runs all the time (e.g., a wall clock), while the virtual time clock runs only when the processor is running in user mode.

In the following code example, `PAPI_get_real_cyc()` and `PAPI_get_real_usec()` are used to obtain the real time it takes to create an event set in clock cycles and in microseconds, respectively [3, 4]:

```

#include <papi.h>

int main(){
    long long start_cycles, end_cycles, start_usec, end_usec;
    int EventSet = PAPI_NULL;

    if (PAPI_library_init(PAPI_VER_CURRENT) != PAPI_VER_CURRENT)
        exit(1);

    /*Create an EventSet */
    if (PAPI_create_eventset(&EventSet) != PAPI_OK)
        exit(1);

    /* Gets the starting time in clock cycles */
    start_cycles = PAPI_get_real_cyc();

    /* Gets the starting time in microseconds */
    start_usec = PAPI_get_real_usec();

    do_some_work();

    /* Gets the ending time in clock cycles */
    end_cycles = PAPI_get_real_cyc();

    /* Gets the ending time in microseconds */
    end_usec = PAPI_get_real_usec();

    printf("Wall clock cycles: %lld\n", end_cycles - start_cycles);
    printf("Wall clock time in microseconds: %lld\n", end_usec - start_usec);
}

```

Possible output:

```

Wall clock cycles: 100173
Wall clock time in microseconds: 136

```